

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 6: **Validasi dan Keamanan Input**

FormRequest, Serangan Umum lewat Input, dan Total yang Dihitung Ulang di Server

Yang Akan Kamu Pelajari

Sejauh ini form yang kamu buat percaya begitu saja pada apa pun yang dikirim penggunaanya. Pertemuan ini mengubah itu:

1. Tiga cara input jahat menyerang aplikasi web (**XSS**, **SQL Injection**, **CSRF**), dan kenapa aplikasi yang kamu bangun sudah aman dari sebagian besarnya
2. Cara menulis **FormRequest** untuk memvalidasi input sebelum controller sempat menyentuhnya
3. Kenapa nilai seperti total transaksi wajib dihitung ulang di server, bukan dipercaya langsung dari input klien
4. Cara menampilkan pesan error dan **flash message** di Blade supaya pengguna tahu kenapa inputnya ditolak

Bagian 1

Dasar Keamanan Input

Kenapa Input Pengguna Tidak Boleh Dipercaya

Setiap kotak isian, setiap parameter URL, setiap field form adalah pintu masuk yang bisa diisi siapa saja, termasuk orang yang sengaja mencoba merusak atau mencuri data. Tiga serangan paling umum lewat pintu ini: **XSS**, **SQL Injection**, dan **CSRF**.

Kabar baiknya: kamu sudah memakai beberapa pertahanan ini sejak Pertemuan 3 dan 5, tanpa tahu persis kenapa itu penting. Tiga slide berikutnya menjelaskan kenapa.

XSS: Cross-Site Scripting

XSS (Cross-Site Scripting): serangan yang menyisipkan kode JavaScript ke halaman lewat input pengguna, lalu kode itu ikut berjalan di browser pengunjung lain saat halaman ditampilkan.

Bayangkan kolom komentar artikel menerima input `<script>` lalu menampilkannya apa adanya ke pengunjung berikutnya, skrip itu ikut jalan di browser mereka, bisa mencuri cookie atau sesi login.

```
{{-- Blade meng-escape otomatis, aman --}}  
<p>{{ $comment->body }}</p>  
  
{{-- Blade TIDAK meng-escape, rawan XSS --}}  
<p>{!! $comment->body !!}</p>
```

Kamu sudah menghindari ini sejak Pertemuan 3: `{{ }}` selalu jadi pilihan default di semua view yang sudah kamu buat.

SQL Injection

SQL Injection: serangan yang menyisipkan potongan perintah SQL lewat input, memanfaatkan query yang disusun dengan menyambung string mentah alih-alih parameter terpisah.

```
// Rawan: input langsung disambung ke string SQL
DB::select("SELECT * FROM users WHERE email = '$email'");

// Aman: Eloquent selalu memakai parameter terikat
User::where('email', $email)->first();
```

Kalau `$email` isinya `' OR '1'='1'`, versi rawan di atas bisa mengembalikan semua baris, bukan cuma satu. Setiap query Eloquent yang sudah kamu tulis sejak Pertemuan 5 otomatis aman dari ini, karena Eloquent selalu mengirim nilai lewat parameter terikat, bukan menyambung string.

CSRF: Cross-Site Request Forgery

CSRF (Cross-Site Request Forgery): serangan yang menipu browser korban agar mengirim request ke aplikasi lain memakai sesi login korban yang masih aktif, tanpa sepengetahuannya.

Bayangkan kamu sedang login ke sebuah aplikasi kasir, lalu membuka situs lain yang diam-diam mengirim form `POST /pos` ke aplikasi itu lewat browsermu sendiri, memakai sesi loginmu yang masih aktif.

```
<form method="POST" action="/pos">
  @csrf
  <!-- ...input lainnya... -->
</form>
```

Blade menyediakan directive `@csrf` untuk menghasilkan token ini otomatis di dalam form, kamu akan mulai memakainya di form kasir pada jobsheet pertemuan ini: Laravel menolak request POST yang tidak membawa token itu.

Bagian 2

FormRequest dan Aturan Validasi

FormRequest: Satpam Sebelum Controller

FormRequest: class Laravel yang membungkus logika validasi dan otorisasi satu request, terpisah dari controller, sehingga controller hanya menerima data yang sudah dinyatakan valid.

```
php artisan make:request StoreArticleRequest
```

Begitu controller menerima parameter bertipe `StoreArticleRequest`, Laravel otomatis menjalankan aturan validasinya lebih dulu. Kalau ada yang gagal, pengguna langsung diarahkan kembali ke form beserta pesan errornya, kode di dalam controller tidak pernah sempat dijalankan.

Menulis Aturan Validasi

```
class StoreArticleRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255'],
            'author_id' => ['required', 'exists:authors,id'],
            'body' => ['required', 'string'],
        ];
    }
}
```

`authorize()` menentukan siapa yang boleh memakai form ini, `true` berarti siapa pun yang sudah lolos ke sini boleh mengisinya. `exists:authors,id` menjalankan query ke database, menolak `author_id` yang tidak benar-benar ada.

Validasi Array: Notasi Bintang

Kalau satu request membawa daftar item, misalnya keranjang belanja, notasi `*` memvalidasi setiap barisnya tanpa perlu loop manual:

```
public function rules(): array
{
    return [
        'items' => ['required', 'array', 'min:1'],
        'items.*.product_id' => ['required', 'exists:products,id'],
        'items.*.qty' => ['required', 'integer', 'min:1'],
    ];
}
```

Setiap elemen di `items` diperiksa satu per satu memakai aturan yang sama. Kalau ada baris yang salah, seluruh request ditolak, dan pesan errornya menyebutkan persis baris mana yang bermasalah, misalnya `items.2.qty`.

Aturan Validasi Kustom Lewat Closure

Kalau aturan bawaan (`required`, `max`, `exists`, dst.) belum cukup untuk memeriksa logika bisnis tertentu, closure bisa ditambahkan langsung di dalam array aturan:

```
'stock' => ['required', 'integer', 'min:0', function (string $attribute, mixed $value, Closure $fail) {  
    if ($value > 1000) {  
        $fail('Stok tidak boleh melebihi 1000 dalam sekali input.');    }  
}],
```

- Closure menerima nama field, nilai yang dikirim, dan `$fail` untuk menolak dengan pesan kustom
- Kalau aturan yang sama dipakai di beberapa form, bungkus jadi class lewat `php artisan make:rule`, supaya tidak ditulis ulang di setiap `FormRequest`

Bagian 3

Total yang Dihitung Ulang di Server

Kenapa Nilai dari Klien Tidak Boleh Dipercaya

Nilai apa pun yang dikirim dari sisi klien, sekalipun perhitungannya terlihat benar di layar, bisa diubah sebelum benar-benar sampai ke server. Field total yang dikirim lewat form HTML bukan pengecualian: ia cuma teks biasa di dalam request, dan tidak ada yang mencegah seseorang mengubahnya sebelum request itu terkirim.

Alpine.js yang kamu pakai untuk keranjang belanja (Pertemuan 3) menghitung subtotal di browser murni untuk ditampilkan, bukan untuk dipercaya sebagai angka final.

Menghitung Ulang di Server

```
$total = 0;

foreach ($validated['items'] as $item) {
    $product = Product::findOrFail($item['product_id']);
    $subtotal = $product->price * $item['qty'];
    $total += $subtotal;

    // simpan baris detail transaksi dengan $subtotal di sini
}

$transaction->update(['total' => $total]);
```

Harga diambil ulang dari database, bukan dari input, lalu dikalikan dengan jumlah yang diminta. Klien boleh mengirim apa pun di field `total`-nya, server tetap menghitung sendiri dan mengabaikannya sepenuhnya.

Bagian 4

Pesan Error dan Flash Message

Menampilkan Pesan Error di Blade

```
<input type="text" name="title" value="{{ old('title') }}">
@error('title')
    <p class="text-sm text-red-600">{{ $message }}</p>
@enderror
```

`@error('title')` otomatis menampilkan pesan itu kalau field `title` gagal validasi, dan tidak menampilkan apa-apa kalau lolos. `old('title')` mengisi ulang input dengan nilai yang tadi dikirim pengguna, supaya form tidak kosong lagi setelah gagal validasi.

Flash Message

Flash Message: pesan singkat yang disimpan di session hanya untuk satu request berikutnya, dipakai menampilkan notifikasi sukses atau gagal setelah redirect.

```
return redirect()  
    ->route('transactions.show', $transaction)  
    ->with('success', 'Transaksi berhasil disimpan.');
```

```
@if (session('success'))  
    <div class="bg-green-50 text-green-700 p-3 rounded-md mb-4">  
        {{ session('success') }}  
    </div>  
@endif
```

Pesan itu muncul sekali setelah redirect, lalu hilang sendiri kalau halamannya dimuat ulang, karena memang hanya disimpan untuk satu request berikutnya.

Menerapkan pada Simple POS

Konsep FormRequest, validasi, dan total di server ini akan kamu terapkan langsung pada Simple POS di jobsheet praktikum: melengkapi model dengan `$fillable` (konsep mass assignment dari Pertemuan 5), menulis `StoreProductRequest` dan `StoreTransactionRequest` untuk model yang sudah kamu bangun sejak Pertemuan 4 dan 5, lalu memastikan total transaksi selalu dihitung ulang dari harga di database, bukan dari input kasir.

Kode lengkap: github.com/se-polinema/simple-pos-ch06

Rangkuman

- XSS, SQL Injection, dan CSRF adalah tiga serangan umum lewat input; Blade (`{{ }}`), Eloquent (parameter terikat), dan `@csrf` sudah jadi pertahanan default sejak pertemuan-pertemuan sebelumnya
- FormRequest membungkus aturan validasi terpisah dari controller, dijalankan otomatis sebelum controller sempat dieksekusi, termasuk validasi array lewat notasi `items.*.field` dan logika kustom lewat closure
- Nilai seperti total transaksi wajib dihitung ulang di server dari data database, tidak pernah dipercaya langsung dari input klien
- `@error()` menampilkan pesan error per field, `old()` mengisi ulang input yang gagal, dan flash message lewat `session()` menampilkan notifikasi sekali setelah redirect

Referensi & Diskusi

Dokumentasi resmi Laravel (Validation, FormRequest), OWASP Top 10

Kode lengkap: `github.com/se-polinema/simple-pos`

Pertemuan berikutnya: Autentikasi & Otorisasi